

# 双栏对照递归与迭代

递归是自顶向下的思考先声明大问题答案再逐步分解为小问题再逐步求解而迭代是自底向上的推进从最小的已知情形出发用循环变量把答案一路滚到目标规模

递归版本通常更短因为控制流交给了调用栈函数体只剩下问题的本质结构。

递归代码的三个检查点基线条件是否存在参数是否严格缩小返回值组合是否正确三者齐备递归就是归纳法缺一个递归就是事故检查点的顺序也有讲究先确认出口再确认收敛最后核对组合。

递归最自然的舞台是树形结构文件系统、抽象语法树在树上递归的形状与数据的形状同构改写成循环反而别扭。树的递归遍历几乎是默认解这一点在任何语言里都成立。

性能方面递归的每层调用都有栈帧开销，深度受限重复子问题若不缓存复杂度可能爆炸递归不是免费的抽象它是用栈空间换表达力的交易深度可达百万的场景，这笔交易不划算。

递归的可读性优势在回溯搜索里最明显八皇后数独求解正则匹配回溯框架天然就是递归的尝试递归撤销三步曲写成递归一目了然写成显式栈则处处是脚手架。

左栏小结递归把如何组合子答案放在聚光灯下把如何调度子问题交给运行时它买来的是思维的经济性代价是栈与深度的约束右栏将给出镜像的论述。

## 右迭代视角

迭代版本通常更长但每一步都摊在纸面上状态转移清晰可见。

迭代代码的三个检查点初始状态是否正确每轮推进是否保持不变量终止条件是否可达这与递归的检查点一一对应只是叙述方向相反递归谈出口迭代谈终点。

迭代最自然的舞台是线性结构数组链表计数循环在顺序数据上循环语句与数据的形状同构滑动窗口双指针前缀和全部以迭代为第一形态。

性能方面迭代没有调用栈负担深度不受语言限制缓存局部性也更好代价是表达分治时需要手工维护显式栈并且循环不变量往往比递归假设更难一眼验证。

迭代的可读性优势在状态机里最明显词法分析协议解析游戏循环状态转移写成配合一张状态表任何人都能按图索骥同样的逻辑硬写递归反而要绕一层解释。

右栏小结迭代把状态如何推进放在聚光灯下调度成本为零它买来的是确定性的资源画像代价是把结构性的美让位给工程性的稳两栏合起来才是完整的工具箱。

第二页继续用同样的双栏结构讨论互相改写的具体手法与各自的边界条件。

## 改写手法左)

递归转迭代的第一步是识别尾位置若递归调用是最后动作直接换成循环累积结果即可阶乘累加链表反转的尾递归版都能这样机械转换转换后行为完全等价

分治型递归则需要把子任务压进显式栈用循环模拟调度弹出一个任务若可直接求解则求解否则把更小的子任务压回栈中。归并排序的迭代版就是这个套路只是循环体取代了调用栈。

改写完成后务必用同一组测试对照两种实现递归版本负责当参照系迭代版本负责通过性能门槛只改不测的改写不算完成，边界输入尤其要覆盖空输入单元素以及触发最大深度的极端规模。

实践中更常见的选择是混合外层用迭代控制轮次内层用递归处理树形子结构编译器构建工具打包器几乎都是这种混合体双栏样本的左栏到此完成它的排版使命。左栏末句递归的深处自有秩序秩序的名字叫归纳。

改写手法有)

迭代转递归是逆操作把循环变量变成函数参数把不变量写进递归假设把终止条件改称基线条件机械但可靠常用于教学演示也用于给老代码补形式化的解释。

注意语言边界没有尾调用优化的运行时里深层递归仍会爆栈此时迭代不仅是风格选择而是必要条件■属于此类■尾调用规范至今没有主流实现。

混合形态在右栏同样成立以迭代驱动主循环遇到嵌套结构就切换到局部递归数据库的查询计划求值器序列化框架的深拷贝都是这种写法的受益者。

最后一句留给右栏迭代的前方步步为营，每一步都踩在不变量上双栏对照样本到此全部收束感谢逐栏读完的你。